Visitor Pattern

Файловая система в памяти / проект и решения

Цель. Отделить структуру файловой системы от независимых операций: суммирования размеров, поиска и статистики. Проект использует Java 17, Maven и собственные синтетические данные; чтение настоящего диска не требуется.

01 / Модель и double dispatch

Element задаёт общий контракт; **FileNode** хранит имя и неотрицательный размер, **DirectoryNode** владеет именованными детьми. Общий sealed-класс **Node** хранит имя и внутреннюю ссылку на родителя. В **Visitor** заранее определены обе перегрузки **visit**.

Первый шаг: динамический тип элемента выбирает его **accept(Visitor)**. Второй шаг: внутри **FileNode.accept** тип **this** известен компилятору, поэтому выбирается **visit(FileNode)**; для каталога - **visit(DirectoryNode)**. Общий обработчик **TreeVisitor** вызывает специализированный **onFile** или **onDirectory**. Проверки типов не нужны.

02 / Один владелец обхода

Обходом владеет **TreeVisitor<R>**: **evaluate(root)** использует явный стек и DFS. Каталог добавляет детей в обратном порядке, поэтому обработка сохраняет порядок вставки. Элементы только выполняют dispatch. Каждый достижимый узел обрабатывается один раз; рекурсивных вызовов нет.

TotalSizeVisitor прибавляет только размеры файлов. **SearchVisitor** объединяет predicate имени и включительный порог **size >= minimumSize**, затем сортирует пути. **StatisticsVisitor** считает файлы, каталоги (включая корень) и максимальную глубину.

03 / Пути, глубина и целостность

Выбранный корень имеет глубину **0** и путь **.**; дети получают глубину родителя + 1. Имя корневого каталога в пути потомков не включается. Для корня-файла результат пути - **..** Разделитель всегда **/**; порядок поиска - лексикографический **String.compareTo**. Пустые каталоги влияют на глубину.

DirectoryNode.add до изменения проверяет повтор имён, владение другим родителем и цикл через цепочку предков. Отрицательные размеры и неоднозначные имена отвергаются; **children()** возвращает неизменяемый snapshot. Переполнение суммы явно вызывает **ArithmeticException**.

GitHub: https://github.com/bigstep832/visitor-pattern-assignment

Проверка поведения

Воспроизводимый запуск / результаты программы и JUnit

Команды выполняются из корня проекта с **pom.xml**. Проверено в CLI на Microsoft OpenJDK 17.0.20.1 и Maven 3.9.11. Установленная IDE не участвует в выполнении тестов.

```
mvn test
mvn package
java -jar target/visitor-pattern-1.0.0.jar
java -jar target/visitor-pattern-1.0.0.jar .png 1000
```

Дерево **campus** содержит README (120), три текстовых файла (800, 200 и 400), изображение (2048), пустой файл и вложенные пустые каталоги. Размеры заданы в байтах. Полный состав находится в **Demo.sampleTree**.

```
Root: campus
Total size: 3568 bytes
Search: suffix=.txt, minimum=300
  docs/architecture.txt
  docs/archive/old-notes.txt
Matches: 2
Files: 6
Directories: 6
Maximum depth: 3
```

Изменённая конфигурация **.png 1000** дала единственное совпадение **assets/logo.png**; общая сумма, количества и глубина остались прежними. Выводы обоих запусков сохранены в **docs/demo-output.txt** и **docs/demo-png-output.txt**.

Автоматизированная проверка

Класс тестов	Тестов	Failures	Errors	Skipped
DemoTest	2	0	0	0
DispatchTest	1	0	0	0
SearchVisitorTest	7	0	0	0
StatisticsVisitorTest	5	0	0	0
TotalSizeVisitorTest	4	0	0	0
TreeIntegrityTest	6	0	0	0

Итого: 25 тестов; failures: 0, errors: 0, skipped: 0. Значения прочитаны из Surefire XML после фактического запуска, а не проставлены вручную.

Покрыты T1-T8: перегрузки, вложенная сумма, оба условия поиска, порядок и относительные пути, статистика, пустые случаи, повторные запуски и недопустимые изменения. Дополнительно проверены 2000 уровней вложенности, переполнение, ошибка predicate и повторный вход. Ожидаемые значения тестов заданы независимо от реализации.

Обоснование решений

Clean Code / границы расширения / повторяемость

Пять конкретных практик Clean Code

1. Содержательные имена. SearchVisitor.minimumSize и Statistics.maximumDepth прямо называют единицы смысла: порог байтов и максимальную глубину; отдельные поясняющие флаги не нужны.

2. Single Responsibility. TotalSizeVisitor.onFile отвечает только за сумму; поиск и статистика выделены в другие классы, а общий обход находится в TreeVisitor.

3. Небольшие методы. StatisticsVisitor.includeDepth содержит одну операцию max и используется для файлов и каталогов, исключая дублирование правила глубины.

4. Инкапсуляция. DirectoryNode.children возвращает List.copyOf, а add является единственной публичной точкой изменения состава; клиент не может обойти проверки коллекцией.

5. Нет цепочек проверки типов. FileNode.accept и DirectoryNode.accept передают this в перегруженный visit; выбор поведения не зависит от instanceof, getClass, reflection или cast.

Повторные запуски и ошибки

Посетитель повторно используется последовательно: каждый **evaluate** вызывает **reset**, а **finally** очищает стек и контекст даже при ошибке. Старые результаты поиска и статистики неизменяемы. Вызов **TreeVisitor.visit** вне **evaluate** и вложенный **evaluate** на том же экземпляре отвергаются. Потокобезопасность не заявляется; predicate не меняет дерево и не вызывает посетителей.

Расширение и компромиссы

Третий посетитель добавлен отдельным commit: новые **Statistics**, **StatisticsVisitor** и его тесты. **FileNode**, **DirectoryNode**, **accept**, **Visitor** и общий обход не менялись. Новая операция требует только нового наследника **TreeVisitor<R>**.

Новый тип элемента, напротив, требует обновить sealed **permits**, контракт **Visitor**, правила обхода и смысл операций для нового типа. Visitor удобен, когда набор типов стабилен, а операций становится больше. Обход выполняет O(n) посещений; поиск добавляет O(k log k) для сортировки. Стоимость построения строк зависит от длины путей.

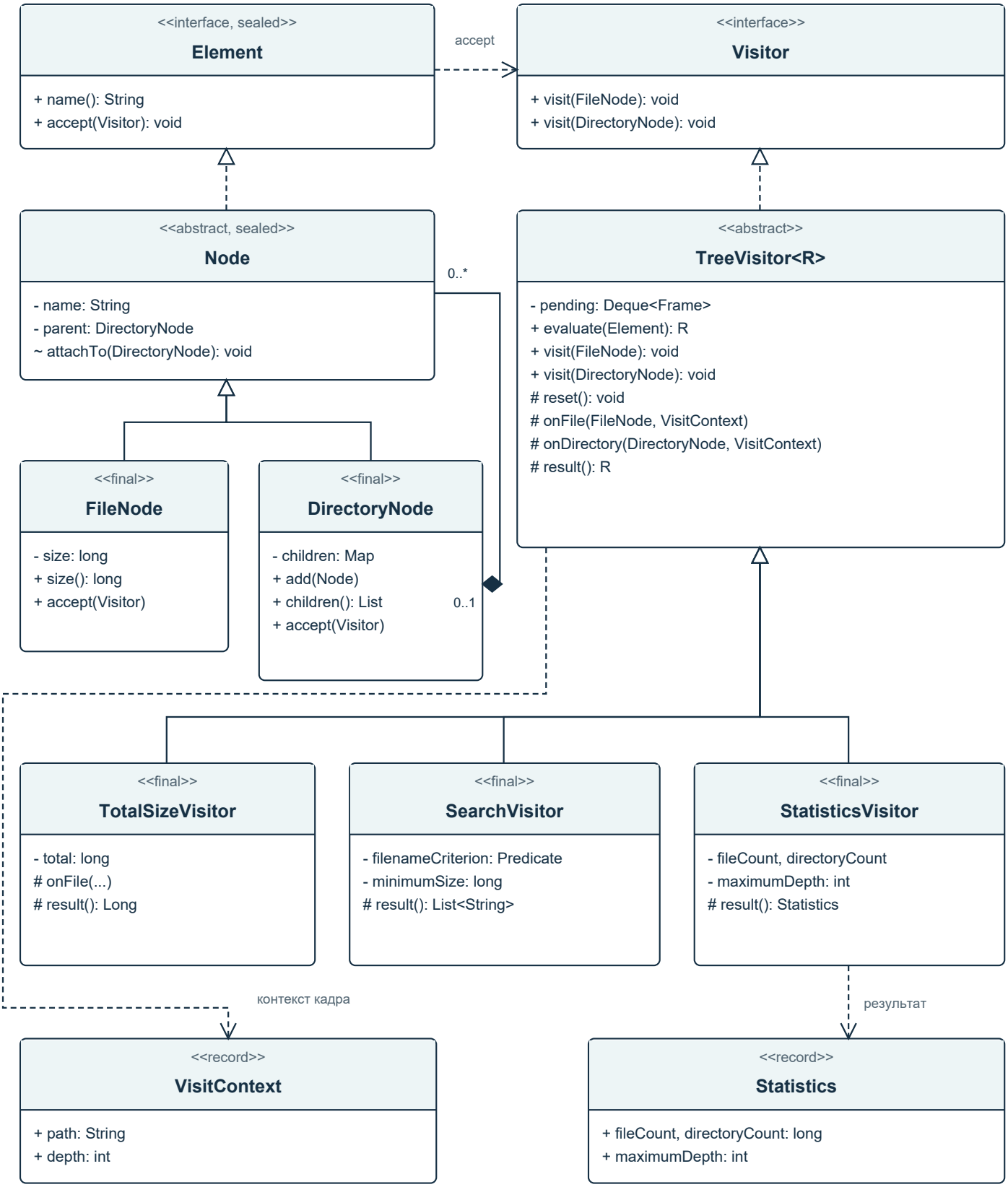
Источники и подготовка

Требования: предоставленный Assignment_6_Visitor_Pattern.pdf, AITU, Assignment 06. Справочные материалы: Java SE 17, JLS 15.12; JUnit 5.10.2 User Guide. Памятка по трассировке и изменению входа: **docs/DEFENSE.md**.

Код, тесты и отчёт подготовлены с помощью ChatGPT/Codex. Это не утверждение о самостоятельном авторстве студента. Перед сдачей следует учесть правила курса о допустимой помощи ИИ и самостоятельно понимать и изменять реализацию.

Структура классов

UML-приложение. Имена и связи соответствуют исходному коду.



Пустой треугольник: наследование / реализация; ромб: владение; пунктир: зависимость.